



FACULDADE DE CIÊNCIAS E TECNOLOGIA

Câmpus Aparecida de Goiânia

Curso de Engenharia de Produção – *Práticas em Engenharia de Produção*

Professor: **Dayvid W P Martins**

Aula de Práticas em Eng. Produção # 04

Arquitetura de Software para Engenharia de Produção

Aula de Práticas em Eng. Produção # 04



1. Por que o Eng. Produção precisa disso?

Dialogar com a TI de igual para igual

2. MVC — Model-View-Controller

O padrão que está em todo sistema industrial

3. Arquitetura em Camadas

A base que sustenta ERPs e MES

4. Hexagonal (Ports & Adapters)

Desacoplamento e integração de sistemas

5. Onion e Clean Architecture

Regras de negócio no centro de tudo

6. Comparativo e Estudo de Caso

Qual arquitetura para cada cenário na fábrica

"Qualquer tolo pode escrever código que um computador entenda. Bons programadores escrevem código que humanos entendem."
— Martin Fowler



Reflexão!



O engenheiro de produção e a
TI

1. Por que
isso
importa?

O Engenheiro de Produção e o Software



Sistemas como ERP, MES, WMS e SCADA são o sistema nervoso da fábrica

O engenheiro de produção é o principal **usuário e especificador** desses sistemas:

1. Define os requisitos de negócio (o que o sistema precisa fazer)
2. Valida se o software atende aos processos reais
3. Participa de reuniões de projeto com a equipe de TI
4. Aprova entregas e homologa funcionalidades

Problema: sem entender a estrutura interna do software, o engenheiro vira refém da TI

O que é Arquitetura de Software?



É a **organização fundamental** de um sistema — como as partes se conectam e se comunicam

Analogia com Eng. Produção:

Produção	Software
Layout da fábrica	Arquitetura do sistema
Fluxo de materiais	Fluxo de dados
Estações de trabalho	Módulos / Componentes
Kanban entre setores	Interfaces / APIs
Inspeção de qualidade	Testes automatizados

→ Assim como um layout ruim gera gargalos, uma arquitetura ruim gera **sistemas frágeis**

Por que o Eng. Produção deve conhecer isso?



- 1. Especificar melhor:** saber pedir um sistema "desacoplado" ou "modular" muda o resultado
 - 2. Avaliar fornecedores:** distinguir software bem arquitetado de "gambiarra" que vai dar problema
 - 3. Reduzir custos de manutenção:** sistemas mal estruturados custam **10x mais** para alterar
 - 4. Integrar sistemas:** conectar ERP + MES + WMS exige entender interfaces e fronteiras
- O engenheiro que **fala a língua da TI** lidera projetos de digitalização com sucesso

"A arquitetura é sobre as decisões importantes — e identificar quais decisões são importantes."
— Robert C. Martin (Uncle Bob)



Observação!



O padrão que está em todo
lugar

2. Padrão MVC

MVC — O que é?

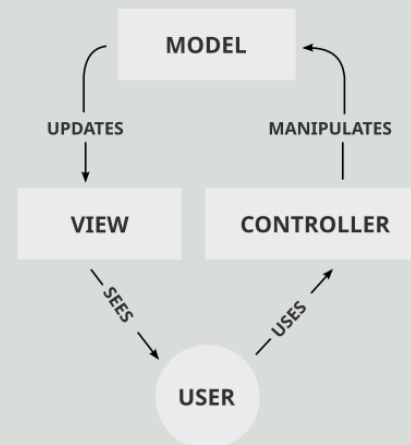


Padrão criado nos anos 1970 que separa o sistema em 3 responsabilidades:

Model (Modelo) → dados e regras de negócio

View (Visão) → interface que o usuário vê

Controller (Controlador) → recebe ações e coordena Model e View





Sistema de Ordens de Produção:

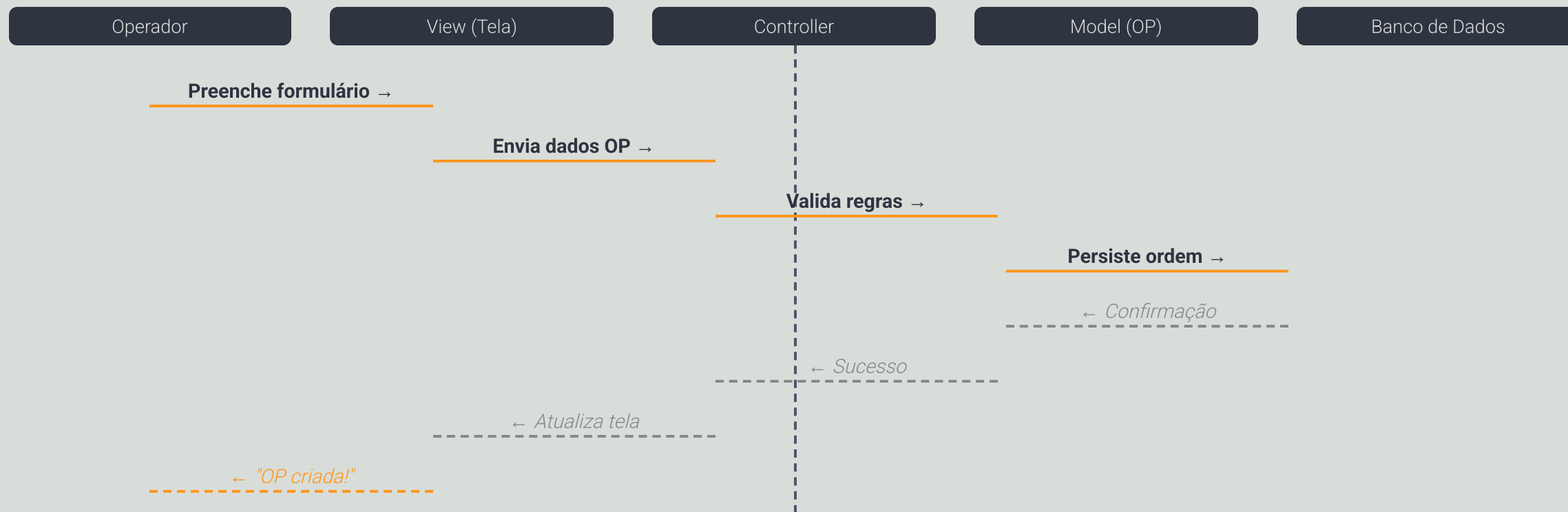
Camada	Responsabilidade	Exemplo
Model	Dados e regras	Ordem, Produto, validação de estoque
View	Tela do operador	Dashboard com ordens do dia
Controller	Lógica de fluxo	"Ao clicar Iniciar, verifica estoque e cria OP"

→ O **operador** vê a View, o **Controller** processa, o **Model** valida e salva

MVC — Diagrama de Fluxo



Cenário: Operador cria uma Ordem de Produção



MVC — Vantagens e Limitações



Vantagens:

- Simples de entender e implementar
- Separação clara entre tela e dados
- Presente em [Django](#), [Spring](#), [Laravel](#), [Rails](#)

Limitações:

- Controller tende a ficar **inchado** ("Fat Controller")
- Regras de negócio se misturam com lógica de apresentação
- Difícil de testar de forma isolada
- Não define claramente onde ficam as [integrações externas](#)

→ Para sistemas simples é suficiente; para sistemas industriais complexos, precisamos de mais

A evolução natural do MVC

3. Arq. em Camadas

Arquitetura em Camadas (Layered)



Organiza o sistema em **camadas horizontais**, cada uma com uma responsabilidade



Regra: cada camada só conhece a camada **imediatamente abaixo**

Camadas — Exemplo ERP Industrial



Camada	O que faz	Exemplo
Apresentação	Interface do usuário	Tela de cadastro de fornecedores
Aplicação	Orquestra casos de uso	"Criar pedido de compra"
Domínio	Regras de negócio puras	"Pedido não pode exceder limite de crédito"
Infraestrutura	Acesso a dados e serviços	PostgreSQL, API dos Correios

→ A regra "limite de crédito" fica no **Domínio**, não na tela nem no banco

Camadas — Por que importa para EP?



Cenário real: A fábrica troca o banco de dados de MySQL para PostgreSQL

Com camadas bem definidas:

- Só a camada de **Infraestrutura** muda
- Domínio e Aplicação permanecem intactos
- Custo da mudança: **baixo**

Sem camadas (código "espaguete"):

- SQL espalhado por todo o sistema
- Cada tela precisa ser reescrita
- Custo da mudança: **altíssimo**

→ Boa arquitetura = **menos retrabalho** = menos custo de manutenção

Camadas — Limitação Principal



O problema: a **direção da dependência**



O **Domínio** (regras de negócio) depende da **Infraestrutura** (banco de dados)

→ Regras de negócio (**estáveis**) acopladas à tecnologia (volátil) — as próximas arquiteturas resolvem isso

Ports & Adapters —
desacoplamiento real

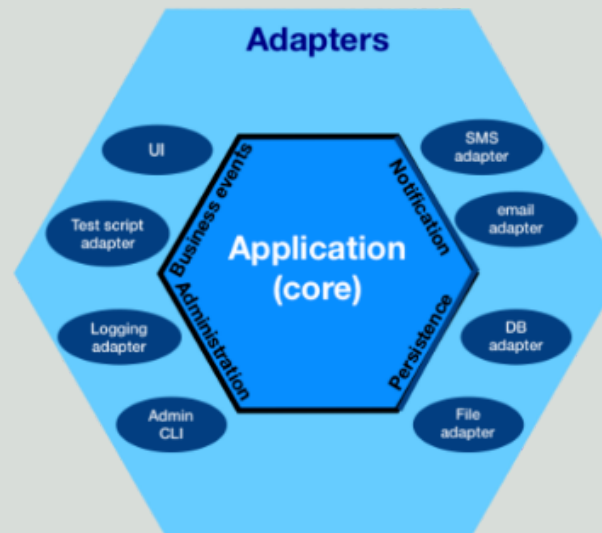
4. Arq. Hexagonal

Hexagonal — O Conceito



Proposta por [Alistair Cockburn](#) (2005) — também chamada "Ports & Adapters"

Ideia central: a aplicação define [portas](#) (interfaces) e o mundo externo se conecta via [adaptadores](#)



Hexagonal — Portas e Adaptadores



Porta (Port): interface que o núcleo define — "eu preciso de X"

Adaptador (Adapter): implementação concreta — "eu forneço X via PostgreSQL"

Exemplo EP — Sistema de Estoque:

Elemento	Descrição
Porta de entrada	<code>RegistrarEntradaMaterial(dados)</code>
Adaptador de entrada	Tela web, leitor de código de barras, API
Porta de saída	<code>SalvarMovimentacao(mov)</code>
Adaptador de saída	PostgreSQL, Oracle, arquivo CSV

→ O núcleo **não sabe** se os dados vêm de uma tela ou de um leitor de barras

Hexagonal — Benefício para a Indústria



Cenário: Fábrica integra leitura de sensores IoT ao sistema de qualidade

Sem Hexagonal:

- Código do sensor misturado com regras de qualidade
- Trocar o sensor significa reescrever a lógica de negócio

Com Hexagonal:

- Sensor é apenas um **adaptador de entrada**
- Regras de qualidade ficam no **núcleo**, intocadas
- Trocar sensor = trocar apenas o adaptador

```
Sensor Vibração → [Adapter] → Porta → Núcleo (regra: vibração > 2mm/s = alerta)
Sensor Novo IoT → [Adapter] → Porta → Núcleo (mesma regra, sem alteração)
```

Regras de negócio no
centro de tudo

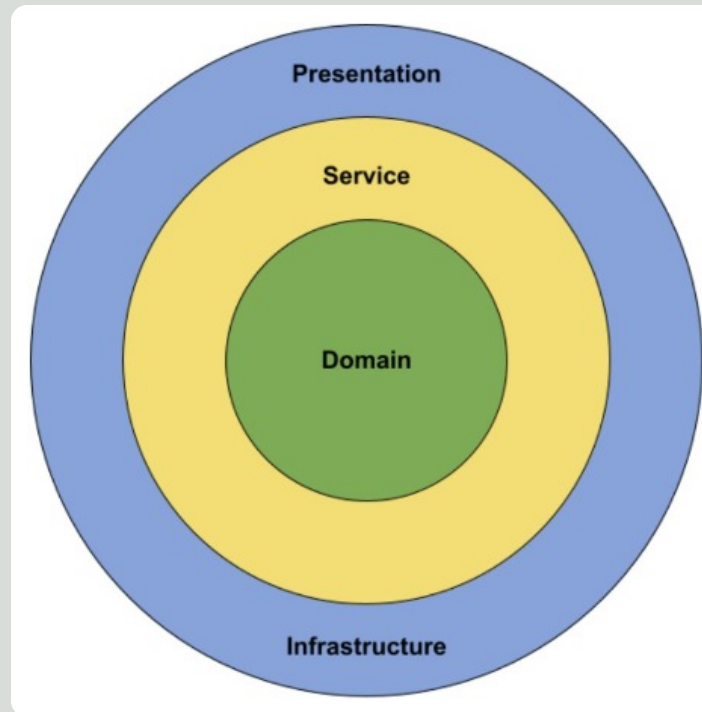
5. Onion Architecture

Onion — O Conceito



Proposta por [Jeffrey Palermo](#) (2008) — organiza o sistema como camadas de uma cebola

Princípio: as camadas externas dependem das internas, [nunca o contrário](#)





Camada	Contém	Depende de
Domínio (centro)	Entidades, regras de negócio	Nada — é autossuficiente
Serviços de Domínio	Lógica entre entidades	Apenas do Domínio
Aplicação	Casos de uso, orquestração	Domínio e Serviços
Infraestrutura (externa)	BD, APIs, UI, frameworks	Todas as camadas internas

Exemplo EP — Regra de estoque mínimo:

A regra `se qtd_estoque < ponto_pedido → gerar alerta` fica no **Domínio**

Não importa se o dado vem do SAP, de um CSV ou de um sensor — a regra é a mesma

Onion vs Camadas Tradicionais



A diferença fundamental está na **direção das dependências**

Camadas Tradicionais:

```
Apresentação → Aplicação → Domínio → Infraestrutura (BD)  
                                ↑ Domínio depende do BD!
```

Onion Architecture:

```
Infraestrutura → Aplicação → Domínio  
                ↑ BD depende do Domínio (via interfaces)!
```

→ A **inversão de dependência** protege o que há de mais valioso: as regras do negócio

A síntese de Robert C.
Martin

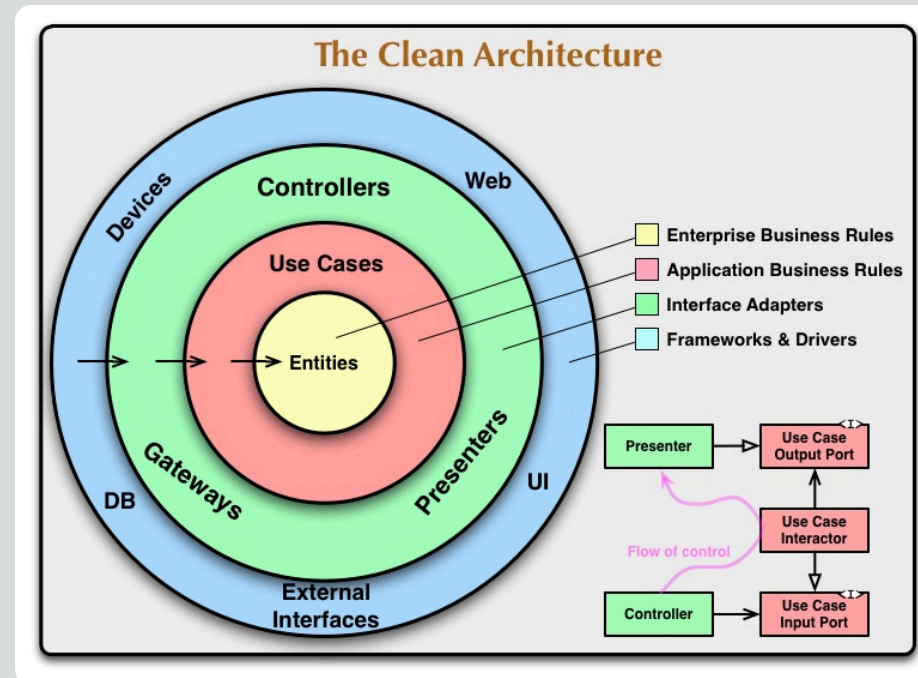
6. Clean Architecture

Clean Architecture — O Conceito



Proposta por **Robert C. Martin** (Uncle Bob, 2012) — sintetiza Hexagonal + Onion

A Regra de Dependência: código-fonte só pode apontar para **dentro** (camadas mais internas)



Clean Architecture — As 4 Camadas

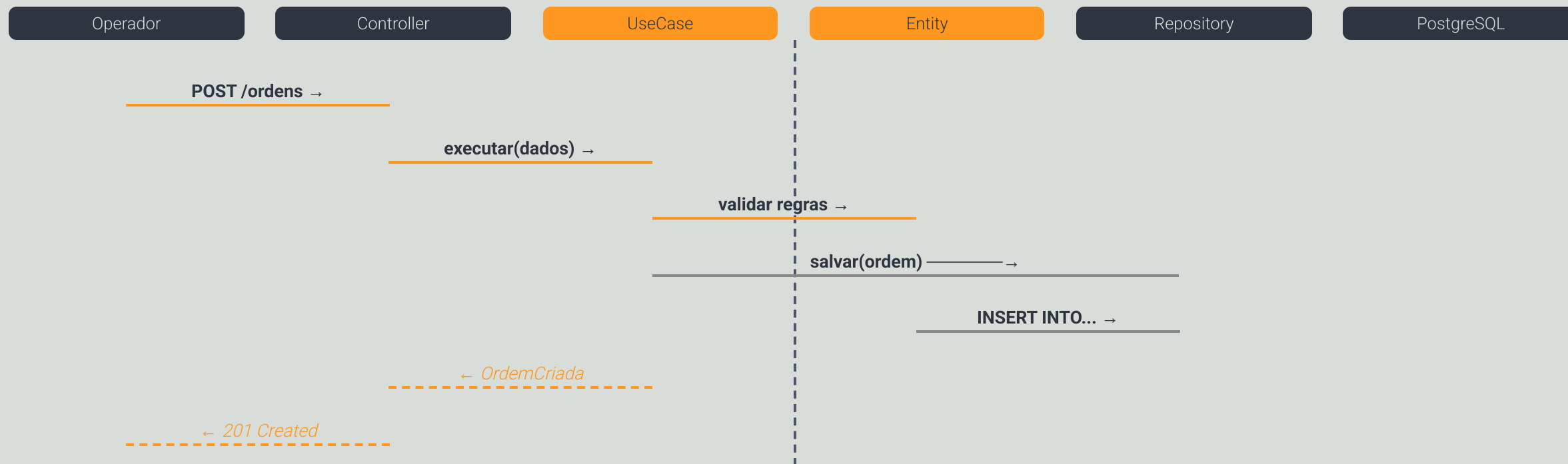


Camada	Responsabilidade	Exemplo EP
Entities	Regras de negócio universais	"OP não pode iniciar sem material"
Use Cases	Regras de aplicação específicas	"Criar OP: validar, reservar material, agendar"
Interface Adapters	Conversão de dados	Controller REST, Repository SQL
Frameworks	Detalhes técnicos	Express.js, PostgreSQL, React

→ As **Entities** são as mais estáveis — mudam apenas quando o negócio muda

→ Os **Frameworks** são os mais voláteis — mudam com a tecnologia

Clean Architecture — Fluxo de uma OP





Entity (regra de negócio pura — não depende de nada externo):

```
class OrdemProducao:
    def __init__(self, produto, qtd, estoque_disponivel):
        if qtd > estoque_disponivel:
            raise RegraViolada("Estoque insuficiente")
        self.produto = produto
        self.qtd = qtd
        self.status = "aguardando"
```

→ A regra "estoque insuficiente" está na **Entity** — funciona sem BD, sem framework



Use Case (orquestra entidades e repositórios via interface):

```
class CriarOrdemProducao:
    def __init__(self, repo_ordem, repo_estoque):
        self.repo_ordem = repo_ordem # interface, não implementação
        self.repo_estoque = repo_estoque

    def executar(self, produto_id, qtd):
        estoque = self.repo_estoque.consultar(produto_id)
        op = OrdemProducao(produto_id, qtd, estoque)
        self.repo_ordem.salvar(op)
```

→ O Use Case depende de **interfaces**, não de PostgreSQL ou MongoDB

Por que "Clean" é importante para EP?



Cenário real: Fábrica troca sistema legado por ERP moderno

Com Clean Architecture:

1. As **regras de negócio** (Entities) são reutilizadas — já estão validadas
2. Apenas os **adaptadores** (Frameworks) são substituídos
3. Os **testes de negócio** continuam passando sem alteração

Impacto financeiro:

Aspecto	Sem arquitetura	Com Clean Architecture
Tempo de migração	18 meses	6 meses
Risco de bugs	Alto	Baixo
Custo estimado	R\$ 2M	R\$ 600K
Regras reescritas	100%	0%

Qual arquitetura para cada
cenário?

7. Comparativo Geral

Comparativo — Todas as Arquiteturas



Critério	MVC	Camadas	Hexagonal	Onion	Clean
Complexidade	Baixa	Baixa	Média	Média	Alta
Desacoplamento	Parcial	Parcial	Alto	Alto	Muito alto
Testabilidade	Média	Média	Alta	Alta	Muito alta
Curva de aprendizado	Fácil	Fácil	Moderada	Moderada	Íngreme
Ideal para	CRUD simples	ERPs básicos	Integrações	Domínio rico	Sistemas críticos

Quando usar cada uma? (Visão EP)



MVC → Sistema interno simples, dashboard de indicadores, protótipos rápidos

Camadas → ERP de pequena empresa, sistema de controle de estoque básico

Hexagonal → Sistema que integra múltiplos dispositivos (IoT, leitores, balanças)

Onion → Sistema com regras de negócio complexas (qualidade, rastreabilidade)

Clean Architecture → Sistema **crítico e de longa vida**: MES de grande fábrica, ERP corporativo

→ A escolha depende da **complexidade do domínio** e da **expectativa de vida** do sistema

Evolução das Arquiteturas — Linha do Tempo



Cada arquitetura surgiu para resolver **limitações** da anterior:

- MVC não separava infraestrutura → surgiu Camadas
- Camadas acoplava domínio ao BD → surgiu Hexagonal
- Hexagonal não formalizava camadas → surgiu Onion
- Clean Architecture unificou tudo com a **Regra de Dependência**

Fábrica de componentes
automotivos

8. Estudo de Caso

Cenário: Modernização do Sistema de Produção



Fábrica de componentes automotivos — 3 linhas de montagem

Situação atual (sistema legado):

- Código monolítico sem separação de camadas
- Trocar o BD exige reescrever 80% do código
- Integrar novos sensores IoT leva meses
- Testes manuais, bugs frequentes em produção

Decisão: Refatorar usando **Clean Architecture**

Estudo de Caso — Modelo Arquitetural



Frameworks & Drivers

React — Painel Operador

Express — API REST

PostgreSQL

MQTT — Sensores IoT



Interface Adapters

Controllers

Repositories

Gateways IoT



Use Cases

Criar Ordem Produção

Registrar Inspeção

Monitorar Sensores



Entities (Domínio)

OrdemProducao

Produto

Inspecao

Estudo de Caso — Resultado da Refatoração



Antes vs Depois:

Métrica	Sistema legado	Com Clean Architecture
Tempo para integrar novo sensor	3 meses	2 semanas
Bugs em produção / mês	15	2
Cobertura de testes	5%	85%
Tempo de deploy	4 horas (manual)	15 min (automático)
Custo de manutenção / ano	R\$ 800K	R\$ 200K

→ A arquitetura não é custo — é **investimento** que se paga em meses

Vocabulário essencial para o
Eng. Produção

9. Glossário e Dicas

Glossário para Reuniões com a TI



Termo	O que significa	Como usar na conversa
Desacoplado	Módulos independentes	"Quero um sistema desacoplado"
Regra de negócio	Lógica do processo produtivo	"Essa regra é do domínio, não da tela"
Interface/Porta	Contrato entre módulos	"Qual a interface de integração?"
Adaptador	Implementação concreta	"Podemos trocar o adaptador do BD?"
Caso de uso	Ação específica do sistema	"O caso de uso 'Criar OP' precisa validar estoque"



1. **Pergunte sobre a arquitetura** antes de contratar um sistema — é tão importante quanto o preço
2. **Exija documentação** dos módulos e interfaces — sistemas sem documentação são armadilhas
3. **Peça demonstração de testes** — se o fornecedor não tem testes automatizados, desconfie
4. **Avalie o desacoplamento** — se trocar o banco de dados exige reescrever tudo, a arquitetura é ruim
5. **Pense em longo prazo** — o sistema vai durar 10+ anos; a tecnologia vai mudar, as regras de negócio não

Prática

10.
Exercícios

Exercício 1 — Identificar Camadas



Dado o seguinte sistema de controle de estoque, classifique cada elemento na camada correta (Entity, Use Case, Adapter, Framework):

1. Regra: "Produto com validade < 30 dias → mover para expedição prioritária"
2. Função: `criar_pedido_compra(fornecedor, itens, prazo)`
3. Tela: Dashboard com gráfico de estoque em tempo real
4. Código: `INSERT INTO movimentacoes VALUES (...)`

Exercício 2 — Projetar Portas



Cenário: Sistema de qualidade que recebe dados de 3 fontes diferentes:

- Inspeção manual (operador digita)
- Sensor automático (câmera de visão computacional)
- Importação de planilha (Excel do laboratório)

Questões:

1. Qual seria a **porta de entrada** comum para as 3 fontes?
2. Quantos **adaptadores de entrada** seriam necessários?
3. A regra "se defeito > 2%, parar a linha" fica em qual camada?



- ✓ **MVC** separa tela, lógica e dados — base para tudo
- ✓ **Camadas** organiza horizontalmente, mas acopla domínio ao BD
- ✓ **Hexagonal** desacopla via portas e adaptadores
- ✓ **Onion** coloca o domínio no centro, protegido
- ✓ **Clean Architecture** sintetiza tudo com a Regra de Dependência
- ✓ O engenheiro que entende arquitetura **lidera a digitalização** da fábrica



Livros:

- MARTIN, R.C. *Clean Architecture: A Craftsman's Guide*. Prentice Hall, 2017.
- FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- COCKBURN, A. *Hexagonal Architecture*. alistair.cockburn.us, 2005.

Artigos:

- Palermo, J. *The Onion Architecture*. jeffreypalermo.com, 2008.
- Martin, R.C. *The Clean Architecture*. blog.cleancoder.com, 2012.

Ferramentas para praticar:

- Draw.io (diagramas de arquitetura)
- Mermaid.js (diagramas em Markdown)
- PlantUML (diagramas UML)



Obrigado! =3

